Branch Assist: presenter learning notes

These notes are a learning guide for the six slides. Read the explanations to understand the work, then practise the short speaking version in your own words. The full notes contain more detail than you should say during the presentation. Use the questions to check whether you can explain a decision without memorising a paragraph.

The visible slides are unchanged. The distinction throughout is between what we implemented, what we observed in the saved tests, and what still needs independent validation. A method being implemented does not mean that it succeeds on every question.

Slide 1 - Generative AI for grounded procedure guidance

The point of this slide

Explain the problem, the solution we built and its overall method. Timing: 0:00 to 2:30. Detailed document processing comes on slide 2; the request pipeline comes on slide 3.

Short speaking version

"I built Branch Assist to help employees find and verify guidance in the eleven supplied procedures. The challenge is that a rule can depend on a situation, a condition or an exception. Finding a familiar phrase is only the beginning.

The solution uses retrieval-augmented generation. In plain language, it searches the procedure collection first, gives relevant passages to a language model, and checks the resulting answer before displaying it. Employees can inspect the cited source, ask a follow-up, narrow the search or flag a response for review.

The collection contains eleven documents and 139 HTML tables, organised into 1,388 searchable passages. The application runs a real local language model; answers are generated when a supported question is asked.

The goal is accurate, relevant, complete and concise guidance with less search effort. We measured engineering behaviour and response diagnostics. Reduced employee effort and independent answer correctness are outcomes still to validate."

Understand the claims on the slide

1. What does generative conversational solution mean? The tool writes an answer to the current question instead of returning a fixed answer from a question-and-answer list. Conversational means a later question can refer to the situation discussed earlier. The interface offers suggested questions for convenience, but those suggestions do not supply canned answers.

2. What does grounded mean? The intended basis for the answer is the supplied procedure text. The model is given retrieved passages and instructed to use only those passages. We also check the draft and show its sources. Grounding is a design and checking approach; it does not guarantee that the model applied the correct condition or included every exception.

3. What is retrieval-augmented generation, or RAG? Retrieval finds relevant material. Augmented means that material is added to the model's input. Generation means the model writes the answer. Think of giving someone the relevant pages before asking them to explain a rule. We did not train the language model to memorise these eleven documents. Document knowledge is supplied at question time.

4. What are the four implemented capabilities? Generative chat produces a draft from source evidence. Follow-up handling resolves the new question using the earlier user situation. The evidence workspace exposes sources, checks and actual execution steps. Quality evaluation compares search methods and records answer diagnostics. Each capability is implemented; none is a claim of perfect performance.

5. What do the numbers count? Eleven is the number of supplied procedure files. The parser counted 139 HTML tables, including nested tables. The 1,388 passages are the smaller searchable units derived from the files. They are not 1,388 independently verified rules. The inventory describes the

collection; evaluation describes how well the system uses it.

What we did and why

We separated document preparation, retrieval, conversation handling, generation and checking. This makes failures easier to locate. If an answer is wrong, we can investigate whether the text was extracted badly, the right passage was missed, a condition was lost, or the model changed the meaning.

We kept original source identity with the derived passages. This allows the employee to inspect where a statement came from. The knowledge collection is limited to the eleven provided Cayman Bank files; linked documents and live policy updates are outside that collection.

We used a local runtime to make the solution runnable on the available CPU without a hosted-model API key. That choice brings a latency tradeoff, discussed on slide 5. The application provides information; it does not execute banking transactions.

Questions I should be able to answer

Why is this useful beyond ordinary search? Search returns passages. This tool also writes a focused response, handles follow-ups and puts source review beside the answer. Search remains essential because the generated explanation must have a traceable basis.

Have we proved that employees work faster? No. That is the business goal to test against the existing search process. We have measured local engineering results, not employee productivity or financial savings.

Does the model know current bank policy? Its usable evidence here is the supplied file snapshot. A publication date does not establish that a document is the current authoritative version.

Evidence and transition

The saved document inventory supports the collection counts. Actual application traces and browser checks support the implemented-capability claims. Say next: "The first engineering step was turning those original files into evidence that search can use without losing its context."

Slide 2 - From original documents to structured, searchable evidence

The point of this slide

Explain how all eleven files became a reliable, inspectable search collection. Follow the four cards from left to right. Timing: 2:30 to 6:00.

Short speaking version

"I treated document preparation as part of answer quality. First, I inventoried the eleven HTML files, including their titles, procedure IDs, publication details and tables.

I then parsed the document structure rather than copying all text into one long string. Headings tell us which section a rule belongs to. Table labels and row conditions tell us when an instruction applies.

Next, I created focused passages. Narrative content targets roughly 110 words, with a 20-word overlap when splitting long blocks. Table continuations repeat the relevant row condition so that an instruction is less likely to become detached from its situation.

Finally, I indexed the same 1,388 passages for keyword and meaning-based search. Each passage retains its original source identity and section. File fingerprints and index checks detect a changed or mismatched local collection.

These choices improve traceability and reduce context loss. They still require review of difficult table branches and exceptions."

Understand each preparation stage

1. Profile: understand the inputs before processing them. The collection contains eleven HTML exports, 79,424 extracted words and 139 HTML tables. That word count comes from the extraction process; it is not a count of unique business requirements. The files vary substantially in size. The business-deposit procedure is much larger than the others, so treating every document as one retrieval unit would produce very uneven results.

2. Extract: preserve meaningful structure. We used BeautifulSoup, an HTML parser, to read the supplied exports. The files contain special classes for policy titles, dates and section headings; relying only on ordinary heading tags would miss structure. We explicitly decode UTF-8, the files' text encoding, to preserve characters. We keep paragraphs, lists and section hierarchy while excluding scripts and display material. We do not follow external links or invent the content behind them.

3. Organise: make focused passages without dropping the situation. A chunk is a smaller piece of source text used for search and model input. Narrative passages are grouped within a section to a target of about 110 whitespace-separated words. When a long block must be split, we prefer sentence boundaries and repeat roughly 20 words across that split. Overlap applies to split blocks; it is not a claim that every neighbouring passage shares exactly 20 words.

4. Index: prepare the collection for search. We save readable passage records with their metadata and create one 384-number embedding for each passage. An embedding is a learned representation used to compare meaning. The same passage collection also supplies the word statistics for BM25 keyword search when the retriever loads. The question-time search therefore compares two ways of finding evidence in the same collection.

Why tables require special care

An instruction in a table often depends on its column heading or leftmost condition. Taking only the action cell can change its meaning. Our parser attaches column labels, carries a cell's condition into later rows when that cell spans multiple rows, and accounts for cells spanning multiple columns. Nested-table text is included once rather than being duplicated by separate traversal.

For an oversized row, continuation passages repeat up to 300 characters of its leftmost condition. An unsplit table row can reach 130 words, and a repeated condition can make a continuation longer. Therefore, 110 words is a narrative target, not a universal maximum or an exact model-token limit. A token is a model's unit of text; the generation stage checks its own token budget separately.

The aim is to keep "when this applies" attached to "what to do." This is an engineering precaution. Text-preservation checks do not prove that every nested table or conditional branch was interpreted correctly.

Traceability and integrity in plain language

Each passage keeps a document ID, title, section, publication text, original filename, passage ID and original-file SHA256 hash. Table passages also keep table and row identifiers. Metadata means these descriptive fields around the passage.

A hash is a digital fingerprint of file bytes. We compare stored fingerprints with the current files and check that the passage store and vector matrix belong together. A changed source or mismatched index stops normal answering until the collection is rebuilt or reloaded. This detects stale or accidentally changed local artifacts. It does not decide which policy version is authoritative or protect against someone replacing all files and their manifests together.

Why these choices and what I would improve

Earlier, larger passages mixed more unrelated instructions into the evidence. Shorter passages were selected after observing development failures. Smaller passages improve focus but can separate an exception from a rule; repeated conditions and limited neighbouring-passage expansion help with that tradeoff. We did not establish 110 words as a universal optimum.

Readable passage files and a small in-memory index suit 1,388 passages. A separate vector database was unnecessary at this scale. Larger collections, frequent updates and document permissions could justify a managed retrieval store later.

Questions I should be able to answer

Why not split every 110 words mechanically? A split can remove a heading, condition or exception. We respect sections and table rows, prefer sentence boundaries and repeat context where needed.

What does 384-dimensional mean? Each passage is represented by 384 numerical values for similarity search. It is not a 384-word passage limit or a confidence score.

Did we extract all the information correctly? We preserved the sources and ran text-preservation and ingestion checks. These support content coverage, but independent review is still needed for the meaning of complex tables and branches.

Evidence and transition

The saved corpus inventory, passage records, source fingerprints and extraction checks support these statements. Say next: "Once this collection is prepared, the next question is how a user request moves through retrieval, generation and checking."

Slide 3 - Retrieve the rule, generate an answer and check the claims

The point of this slide

Explain one request from left to right, then explain the reasons for the main technology choices. Timing: 6:00 to 9:30. Read the numbered steps as an engineering explanation; use the short version when presenting.

Short speaking version

"A question first passes through context and scope handling. The retriever then combines keyword search with meaning-based search and reranks the best candidates against the question.

We give the language model a small evidence pack: up to three passages, targeting about 520 words. Qwen3 runs locally and writes a concise answer using those passages and source labels.

Before release, the system checks the citations, numbers, selected threshold wording and whether statements appear supported by the cited text. A failed draft can receive one correction using the same evidence. If it still fails, the generated draft is withheld and the tool shows original passages for review.

RAG gives us replaceable, traceable source material. Hybrid search combines exact terms and paraphrases. Local quantized inference fits the available machine, with slower CPU responses as a tradeoff. The checks make behaviour inspectable, but they do not certify correctness."

Walk through what we actually implemented

1. Understand the question and scope. The resolver looks for a procedure ID, a named topic or a follow-up referring to the earlier user situation. A selected procedure restricts the search. Basic input checks bound the request and catch a small set of obvious instruction attacks or identifier patterns. These are limited rules, not comprehensive security or privacy protection. The source and index snapshot is checked before supported retrieval.

2. Search in two complementary ways. BM25 ranks passages using matching terms, how distinctive those terms are in the collection and passage length. It helps with exact procedure IDs, amounts and specialist language. MiniLM embedding search compares the question's representation with the saved passage representations. It helps when the question paraphrases the document. Both methods can return an irrelevant passage, so we measure them rather than assuming either is sufficient.

3. Combine rankings and rerank. Each search supplies up to 30 candidates. Reciprocal rank fusion, or RRF, combines their ranking positions. It avoids pretending that a BM25 score and a similarity score use the same scale. Up to 30 fused candidates are then scored by the MiniLM cross-encoder, which reads the question and each passage together. That second, more detailed comparison reorders the shortlist. It adds CPU work but can improve which result appears first. Embedding search uses separately prepared representations; a cross-encoder makes a joint question-passage comparison.

4. Decide whether there is enough retrieval support and select evidence. The runtime obtains eight ranked hits, can add a limited adjacent qualification or continuation for the leading two, removes near-duplicates and packs at most three complete passages. It targets 520 words rather than cutting through a trailing exception. An oversized first passage can exceed that word target; the later token-budget check still applies. The top-hit cosine threshold of 0.24 and reranker threshold of -2.0 are development heuristics. They are not calibrated probabilities that the evidence or answer is correct.

5. Generate an answer. The generator is Qwen3-4B using a Q4_K_M GGUF file through llama.cpp.

Qwen3 is the model family; 4B describes its approximate parameter count. Quantization stores model weights more compactly, here using a four-bit configuration with mixed precision. GGUF is the saved model format; llama.cpp is the inference engine that runs it locally. We use existing pretrained weights, not a model fine-tuned on these documents. The runtime uses the CPU without a hosted inference API. It supplies the question, clearly labelled evidence and instructions to preserve conditions, stay concise and cite the supplied sources.

The input limit is 3,800 tokens and generation is limited to 240 new tokens. The prompt requests one to three sentences and at most 120 words; the mechanical checker rejects drafts over 190 words. These are different controls: requested style, generation budget and rejection limit. Greedy decoding favours repeatability but does not remove hallucinations. The bounded answer task uses non-thinking mode; the interface does not display a private reasoning transcript.

6. Check the draft against its citations. Mechanical checks look for valid source labels, attribution for substantive claims, numeric values supported by the cited passage, selected strict-versus-inclusive threshold changes, and excessive length. A number appearing somewhere in the evidence pack is insufficient if the claim cites a different passage. Equivalent forms such as 36 and 36.00 are normalised for comparison.

When those checks pass, support screening examines the cited text. A substantive verbatim match gets a textual-support result. Other claims use a DeBERTa natural-language-inference model, or NLI. NLI compares a passage and a statement and classifies support, contradiction or neither. We screen paraphrases using support at least 0.55 and contradiction below 0.20, including shorter source windows. Those model outputs are not probabilities that a bank instruction is correct. Both verbatim matching and NLI can miss whether the rule applies to this particular situation.

7. Release, correct once or show sources. A passing draft is displayed with citations. A failed draft can get one correction containing failed-check guidance and the same evidence. We do not loop indefinitely or fetch new evidence during that correction. If the revised draft still fails, the answer is labelled source-only and shows retrieved excerpts. If the model explicitly reports insufficient evidence, the system reports that instead of inventing an answer.

Why this architecture

RAG makes procedure replacement and source review explicit. Fine-tuning alone would not give reliable citations or source-version control. Sending all eleven documents for every question would consume context and introduce unrelated rules. Search alone is useful as a fallback, but the requested solution also needs generated conversational answers.

The local models and small in-memory index make this runnable on the available laptop. They also limit answer quality, throughput and latency. The workflow has a bounded sequence and one possible correction; it is not an open-ended agent that performs banking actions.

Questions I should be able to answer

Why not let the same language model judge its own answer? We use mechanical rules and a separate support model as additional signals. That reduces reliance on a single unchecked draft, but an independent human assessment is still necessary.

Can all checks pass and the answer still be wrong? Yes. The right words can describe the wrong situation, an exception may not have been retrieved, or a support model can misclassify. Passing means the implemented screens passed, not that factual correctness was established.

Why only one correction? It bounds latency and complexity. A repeated attempt with the same incomplete evidence cannot guarantee a better answer, so the explicit source-review outcome is important.

Evidence and transition

The implemented request path, saved response checks and activity records support this explanation. Say next: "The user experience makes these stages useful by keeping the conversation and its evidence together."

Slide 4 - Context, citations and review in one conversational workflow

The point of this slide

Explain how a person uses the tool, how follow-ups are resolved and what the recorded dialogue demonstrates. Timing: 9:30 to 13:00.

Short speaking version

"The workspace keeps the question, the answer and its evidence together. Employees can search the procedure library, select a scope, ask a question and open the supporting passage from its citation.

The recorded dialogue first asks the overnight coin-machine limit and receives $5,250. The follow-up asks about monthly testing and receives single control. The important behaviour is that the second turn retrieves evidence for the new task while keeping the relevant procedure context.

We use earlier user questions to resolve context. We do not treat earlier generated answers as source facts. A changed condition retains the original situation; a clearly named new topic starts a new search. Short ambiguous requests can trigger clarification.

Employees can inspect source text, output checks and actual activity, flag a response and export the investigation. The workflow supports review, while the resolver and support checks still have known limitations."

Understand the three experience claims

Search and scope. The procedure library lets a person find one of the eleven documents and restrict retrieval to it. All-procedure scope allows topic changes across the collection. If the question explicitly names a procedure different from a manually selected scope, the tool asks the person to change that selection. It does not silently ignore the restriction. Guided starters and suggested follow-ups are navigation aids; supported answers still run retrieval and generation.

Follow-up resolution. A follow-up often leaves something unsaid: "What if they are present?" only makes sense with an earlier situation. We implemented readable rules using procedure IDs, recognised topic words, pronouns and follow-up phrases. The resolved query includes the earlier user situation when appropriate. This is explicit context handling, not an LLM that understands every possible conversation.

Inspect and review. Citation controls open the supplied passage in the Evidence desk. Evidence shows original text and document details. Checks shows attribution and output screening. Activity shows actual recorded stages, decisions and durations. Review feedback is stored in the current session, and exports include the conversation, sources, checks, activity, selected scope and feedback. A review flag does not automatically retrain a model or create an external case-management ticket.

Three different conversation situations to understand

A new task within the same procedure. The coin example changes from overnight storage to monthly testing. We retain the relevant procedure context and retrieve fresh testing evidence. If the follow-up explicitly names a new task, unnecessary wording from the old task can be omitted from the retrieval query. This prevents "overnight" from overwhelming the search for testing instructions.

A changed condition within the same situation. After asking about an emailed notary document, "What if the requester is physically present?" changes one condition. The emailed-document detail

must remain. The resolver also preserves the situation when that hypothetical repeats the same notary topic or procedure ID. PR1000 still prohibits the emailed-document scenario even with physical presence.

A new topic. A question explicitly naming savings-bond redemption after a coin discussion starts a new topic. Old coin details should not be carried into the query. Every turn retrieves current evidence from the supplied snapshot; a previous generated answer is never treated as proof.

What the recorded dialogue does and does not demonstrate

The two answers on the slide reproduce captured application outputs: $5,250 for overnight storage and single control for monthly testing. These source facts are different from the weekly-balancing and random-monthly-cash-count rules, which use dual control. The example illustrates why identifying the exact task matters.

S1 and S2 are the labels shown with those recorded messages. Source labels are local references to passages supplied with a response, not permanent document IDs or an always-increasing global sequence. A later live run can assign labels differently. Inspect the underlying procedure and passage rather than expecting every follow-up to use S2.

This example demonstrates a particular successful workflow. The original follow-up evaluation still has two misses out of four fact checks; one good captured conversation does not establish general conversational accuracy.

Session behaviour and limits

Each browser session owns its messages, topic history and feedback. The app passes up to four recent context turns; resolution primarily uses the latest resolved user situation with a bounded text window. Model and index objects can be reused, but another browser session does not inherit this conversation. This is session isolation, not enterprise user authentication.

New investigation clears the conversation. Changing scope clears the topic history. Unsupported or refused topics clear the active UI topic history, reducing accidental carryover. Ambiguous entities, subtle topic changes and long conversations can still be misresolved. The reset, scope and clarification controls give the person a way to recover.

Questions I should be able to answer

Why not send the entire chat history to the model? Previous generated text may contain an error and unrelated topics can distract retrieval. We use bounded user-question context to resolve the new question, then retrieve fresh source evidence.

Does Activity reveal the model's reasoning? It shows observable system actions, selected evidence, checks and decisions. It does not expose or invent a private chain of thought.

Does feedback improve the model immediately? No. It records a review signal in the session and export. A governed review and improvement process would be future work.

Evidence and transition

Captured response records support the dialogue. Browser integration checks cover source selection, feedback/export, scope, reset and independent sessions; conversation regressions cover the resolver rules. Say next: "A useful interface still needs measured quality, so I separate evidence retrieval from the quality of the written answer."

Slide 5 - Evaluate retrieval and response quality as separate measures

The point of this slide

Explain every displayed number in ordinary language, then explain how to judge the four requested quality dimensions. Timing: 13:00 to 17:00. This slide is about interpreting evidence, not presenting one accuracy percentage.

Short speaking version

"I evaluated retrieval and written answers separately. Across thirty procedure questions, keyword search found a labelled relevant passage in its top five results for twenty-seven cases. Semantic search did so for twenty-eight, and hybrid retrieval with reranking did so for all thirty.

That does not mean thirty correct answers. The expected-fact check passed twenty-three of thirty ordinary answers. It is phrase-based, so it is a useful regression signal rather than independent factual accuracy.

The slide also makes the weaknesses visible: six of thirty-five generated evaluations ended with source excerpts, and two of four original follow-up fact checks passed. Local CPU response time was about fourteen seconds at the median and twenty-four seconds at the ninety-fifth percentile, excluding model loading.

For the next validation stage, two independent reviewers should score accuracy, relevance, completeness and conciseness on unseen scenarios, with critical errors and answer coverage reported separately."

How the evaluation was organised

The saved suite contains 30 single-turn procedure questions, four two-turn conversations and six guardrail scenarios. The 30 procedure questions include 19 development and 11 challenge cases. AI helped draft the cases; their source references were checked. They are not an independently authored benchmark. Once challenge questions have been inspected during development, they are not a blind holdout.

We compared three retrieval configurations on the same 30 questions and collection: BM25 keyword search, MiniLM semantic search, and hybrid retrieval plus cross-encoder reranking. We retained retrieved passage identities, outputs, source text, checks and timing. This allows review of individual failures rather than relying only on summary figures.

Understand each number on the slide

27/30, 28/30 and 30/30: relevant evidence in the first five search results. A hit means at least one result matches the labelled supporting passage or an accepted equivalent. It does not mean the complete answer, all conditions or every exception reached the generator. The live pipeline subsequently selects at most three passages from a wider ranked shortlist. Search can succeed while evidence selection or generation still fails.

If asked about MRR, or mean reciprocal rank, explain that it rewards finding the first relevant passage earlier: first place contributes 1; second place contributes one-half; a miss in the first five contributes zero. The hybrid MRR@5 is 0.9278. It describes ordering, not correctness of the generated answer.

23/30: the expected-fact proxy. Proxy means an indirect measure. For each required fact group, the

check looks for one accepted phrase in the displayed generated answer. Passing means all its required groups matched. A correct paraphrase can fail, and a wrong answer containing the expected words can pass. A withheld draft or source-only result does not count as a successful generated answer. Therefore, do not describe this as 76.7% independently established accuracy.

6/35: source-only outcomes among generated evaluation records. The model was invoked, but the final draft did not pass the required checks after the permitted correction. The interface displayed retrieved excerpts for review. This is a useful failure outcome to expose, but it also means the tool did not deliver an accepted generated answer for those cases. It is not evidence that all six drafts were factually wrong; support screening can reject correct text.

2/4: original follow-up fact-proxy passes. Two of the four retained final-turn answers matched all expected fact groups. The other two misses remain in the report. The successful dialogue on slide 4 and later targeted resolver checks do not replace or improve this recorded number automatically.

28 words: average checked draft length. This mean covers 32 generated records with word-count diagnostics, including drafts later withheld. It is not the average length of only successful user-visible answers. A short answer can omit an essential condition, so length is a diagnostic for conciseness, not a quality verdict.

13.8 seconds p50 and 23.9 seconds p95: local end-to-end timing. p50 is the median: about half the measured generated evaluations completed within that time. p95 is the approximate ninety-fifth percentile of the observed sample. These times include the measured request stages and any correction, but exclude cold model loading. They describe serial local CPU measurements, not a guaranteed response time or concurrent service capacity.

How I would test the four requested dimensions

Accuracy: is each instruction correct for this exact situation? Compare claims, amounts, deadlines, permissions and prohibitions with the original source. Automatic citation, number and support checks help identify risks. Independent reviewers must judge meaning and applicability. Mark a reversed prohibition, invented permission or wrong monetary limit as a critical error even if the answer is otherwise well written.

Relevance: did we answer the question that was actually asked? A retrieved passage can be on the right general topic while the answer explains the wrong task. Reviewers should judge whether the response directly addresses the requested fact and avoids unrelated workflow steps. This needs answer-level review in addition to retrieval ranking.

Completeness: did we preserve material conditions and exceptions? A correct base amount can still be incomplete if the exception changes the outcome. The required-fact proxy helps with known regression cases, but reviewers need to examine applicability, warnings and omitted qualifications against the source.

Conciseness: is the answer brief without losing meaning? Word counts, prompt instructions and token limits constrain length. Reviewers should decide whether the response contains necessary context without repetition. A one-word answer is not automatically better than a two-sentence answer that includes the governing condition.

What stronger validation would look like

Two subject-matter reviewers independently score each dimension from 0 to 3 using the original procedures. A further reviewer resolves substantive disagreements. Use unseen scenarios, paraphrases, long conversations, conditional tables and high-impact exceptions. Keep critical errors separate from average scores.

Report quality among answered questions and coverage across all eligible questions. Otherwise, a

system could appear accurate by refusing most requests. Review correct abstentions, unnecessary refusals, source-only fallbacks and incorrect generated answers separately. Compare employee task success and time to correct guidance with existing search.

The recorded 49 passing automated tests check software behaviour such as conversation rules and artifact integrity. They are not 49 independently validated banking answers. Browser checks show that the workflow executes. Human review establishes whether the substantive guidance is correct and useful. These are different kinds of evidence.

Questions I should be able to answer

How can retrieval be 30/30 but the fact proxy only 23/30? Finding one relevant passage is an earlier, easier requirement. Selection can omit an exception, generation can change a condition, checks can withhold a valid draft, and phrase matching can miss a good paraphrase.

Is the system ready for operational use? The engineering workflow and small regression results are demonstrated. Independent quality review, broader testing and deployment controls remain future work. The notes and slide do not claim those have been completed.

Evidence and transition

The accepted metrics, raw response records, retrieval results and independent-review rubric support these explanations. Say next: "The demonstration makes these distinctions visible through ordinary questions, changed conditions and an exception that requires source review."

Slide 6 - Demonstrate answers, follow-ups and evidence review

The point of this slide

Explain what each demo scenario proves, what to inspect and how to describe an imperfect outcome accurately. Timing: 17:00 to 20:00, followed by the separate 15-minute walkthrough.

Short speaking version

"The demonstration follows three situations. First, an ordinary coin-machine question and a follow-up show generated answers with fresh source evidence. Second, the notary example changes a condition and checks that the original restriction remains attached to the situation. Third, the savings-bond example introduces an exception and shows why inspecting source text matters.

During the walkthrough I will open citations, review checks and activity, and show clarification or source review when an accepted generated answer is unavailable. The approved silent recording includes an exception draft that remains withheld after one correction; that is a visible limitation, not a successful generated answer.

The next engineering steps are independent quality review, broader exception testing and deployment controls. AI assisted development and test drafting, and the local model generates runtime answers. The outputs were checked through source inspection, software tests, saved traces and browser workflows; those checks are distinct from independent subject-matter validation."

Scenario 1: a routine rule and a new task

Ask: "What is the overnight storage limit for a coin-counting machine?" Inspect PR63 for the maximum of $5,250 in a single machine. Then ask: "And is monthly testing done in single or dual control?" Inspect the testing passage for single control.

What this demonstrates: the tool generates a focused answer, attaches source references and retrieves different evidence for the follow-up. It should retain the relevant procedure context without answering the old storage question again. Do not generalise the testing control to weekly balancing or random monthly cash counts, which use dual control.

Scenario 2: a changed condition must not reverse a prohibition

Start a new investigation. Ask: "Can a notary notarize an emailed document?" Then: "What if the requester is physically present?"

PR1000 prohibits the emailed-document scenario even when the requester is present. The second question changes presence; it does not remove the fact that the document was emailed. Open the source and inspect the resolved context or activity. The intended behaviour is to preserve the governing situation rather than infer that presence creates permission.

If the generated answer is withheld, describe that actual outcome and show the passage. Do not call it a successful generated response or improvise the rule from memory.

Scenario 3: the exception is part of the answer

Start a new investigation. Ask for the savings-bond redemption limit for an account open less than twelve months. Follow up by asking about eligibility for The Private Bank Experience.

PR1949 gives a $1,000 daily aggregate redemption-value limit for the newer-account situation and an exception for eligible Private Bank Experience customers regardless of account age. The exception does not remove every other condition in the procedure. Inspect the applicable source instead of turning it into a blanket exemption.

In the approved recording, the exception draft fails a required check after one correction, and the tool shows the original passages. Explain both sides: the evidence remains available, but the model did not deliver an accepted generated answer for that turn. Passing all the expected demo scenarios is not a claim we should make.

Additional behaviours to show in the walkthrough

Keep All 11 procedures selected for a topic change. After a coin question, ask about savings-bond redemption fees and inspect the switch to PR1949, which prohibits a bank redemption fee.

After resetting, ask "Can they do that?" to show a missing-context clarification. Ask about tomorrow's weather or an unavailable procedure to show the boundary of the document collection. Inspect Evidence, Checks and Activity; flag a response and export the investigation. These actions demonstrate the review workflow, not external transaction execution.

Warm the local models with one question before presenting, then reset. This avoids confusing cold-start loading with ordinary response time. Identify saved playback as a recording. The approved silent demo lasts 9 minutes 25 seconds, leaving time within the 15-minute segment for source inspection and discussion. The approved recording itself has not been changed.

What the next engineering steps mean

Independent quality review means obtaining new, independently assessed scenarios and adjudicating critical errors, not simply rerunning the same phrase checks.

Broader exception tests means covering more branches, multi-turn changes, cross-references and failures where a base rule is retrieved without its exception. Stronger approved models should be compared on the same scenarios, with latency and answer coverage measured together.

Deployment controls means authoritative source versions, employee authentication, document permissions, monitoring, escalation, retention rules and rollback. These are future requirements. Local inference and a review button do not establish an enterprise operating model.

Questions I should be able to answer

What did AI help with? Requirements analysis, design, implementation, test-case drafting and presentation preparation. Runtime generation is a separate use of AI. The source-checked regression labels are not independent ground truth.

What was validated? Source and index consistency, software behaviour, recorded request execution, browser interactions and the saved evaluation diagnostics. Independent reviewers still need to judge factual accuracy and operational usefulness.

What is the strongest conclusion I can defend? A working, inspectable generative procedure assistant has been built, with measured retrieval and answer diagnostics and explicit failure outcomes. The evidence supports further validation and improvement; it does not establish perfect answers or operational approval.

Handoff

Say: "I will now show the working tool, starting with a routine question and then inspecting the evidence behind its follow-up."